

Sistemas Operativos 2026



Clase 2 U2 Procesos

Ing. Juan Manuel Calvo

¿Qué es un proceso?

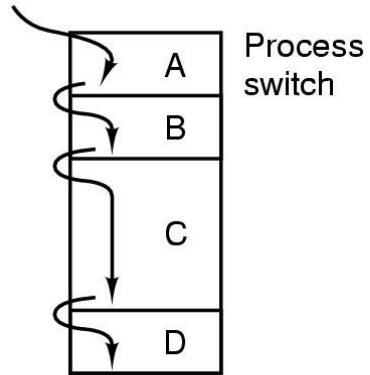
- Un "programa ejecutándose" con su contexto asociado (datos y ambiente).
- no es lo mismo que un "programa" o "aplicacion", un dado programa puede estar corriendo 0, 1 o más veces.
- Cada invocación de un programa es un proceso separado, con su propio espacio de direcciones y contexto.
- Algunos sistemas operativos oscurecen esta distinción al usuario. Ej: ¿que sucede cuando Ud. hace doble click sobre el ícono del browser por primera vez? y ¿por segunda vez?.

El modelo de procesos

- Para ejecutar el código de los programas en una computadora este se organiza en procesos secuenciales.
 - Cada proceso es una ocurrencia de un programa en ejecución, y tiene su propio:
 - Contador de programa
 - Registros
 - Variables
 - Cada proceso puede estar detenido o ejecutándose en una CPU
 - Las CPUs reales están conmutando continuamente entre estos procesos
 - Se puede asumir que los procesos están corriendo en paralelo.
- Esta conmutación rápida de procesos es una forma de multiprogramación.
- Generalmente la secuencia en la que se ejecutan de procesos no es predecible.

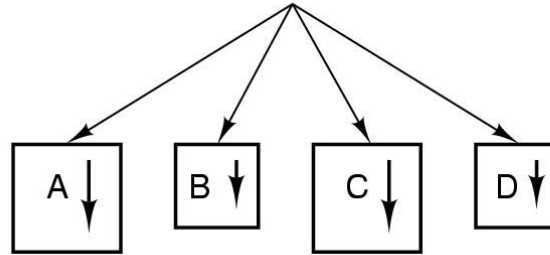
Multiprogramación – una sola CPU

One program counter

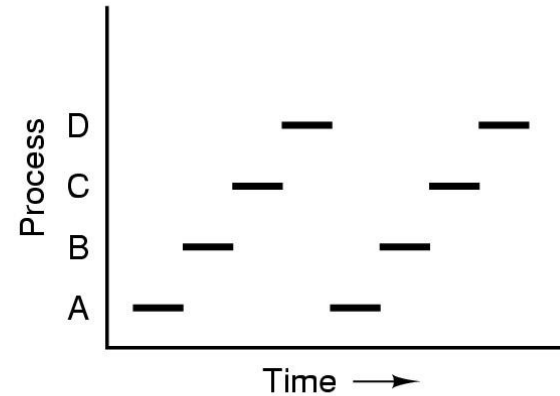


(a)

Four program counters



(b)



(c)

- a) Multiprogramación de cuatro programas.
- b) Modelo conceptual de cuatro procesos secuenciales e independientes
- c) Sólo un programa está activo a la vez

Procesos/Tiempo compartido

- Permite a los programas actuar como si fueran "dueños" de la maquina.
 - No hay que preocuparse por otros programas que estén corriendo
 - Comparten los recursos
 - Obtienen los servicios del Sistema Operativo
- Se aprovecha de dos mecanismos del hardware/CPU
 - Memoria relocacionable
 - estados de "kernel mode" y "user mode", "traps".

Objetivos:

- **Multiprogramación:** tener en todo momento un proceso para ejecutar de tal manera de maximizar la utilización de la CPU.
- **Tiempo compartido:** cambiar un núcleo de CPU entre procesos con tanta frecuencia que los usuarios puedan interactuar con cada programa mientras este se está ejecutando.
- El **planificador de procesos** selecciona un proceso de los disponibles para ser ejecutado en un núcleo de CPU.
 - Cada núcleo de CPU puede correr un sólo proceso a la vez.

Memoria relocacionable

- Es una funcionalidad de la CPU que permite a los programas ser contruidos sin necesidad de saber donde se van a ubicar en la memoria y también evitar pisar la memoria de otro programa por error.
- Se puede implementar de varias maneras (segmentación, memoria virtual).
- Cada programa que está corriendo tiene su propio espacio de direcciones privado.

"Supervisor mode" / "User mode" and "traps"

- Cuando la CPU está en el modo supervisor, puede ejecutar cualquier instrucción y puede direccionar cualquier parte de la memoria.
 - El núcleo del sistema operativo corre en el modo supervisor y puede cambiar a modo de usuario en cualquier momento.
- Cuando la CPU esta en modo de usuario, tiene solamente acceso a su propio espacio de direcciones, y no puede hablar directamente a los dispositivos.
 - Los procesos de usuario corren en modo de usuario, para conmutar al modo supervisor se hace por medio de un trap.
 - En el modo de usuario ciertas instrucciones del procesador no se pueden usar directamente, todo intento de usarlas causa un trap.
 - Instrucciones restringidas: aquellas que manejan el hardware: las instrucciones de entrada/salida. Las que comandan la MMU, etc.
 - Modo supervisor no tiene nada que ver con el usuario Administrador o el root.

Creación de procesos

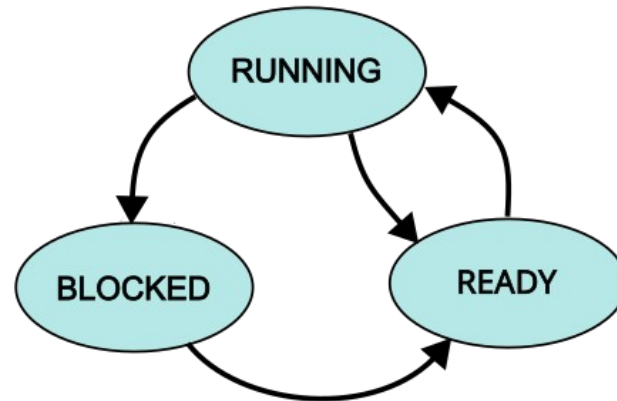
- Inicialización del sistema
- Ejecución de una llamada al sistema por un proceso en ejecución.
- Pedido de un usuario de crear un nuevo proceso
- Arranque de una tarea por lotes (batch).

Finalización de un proceso

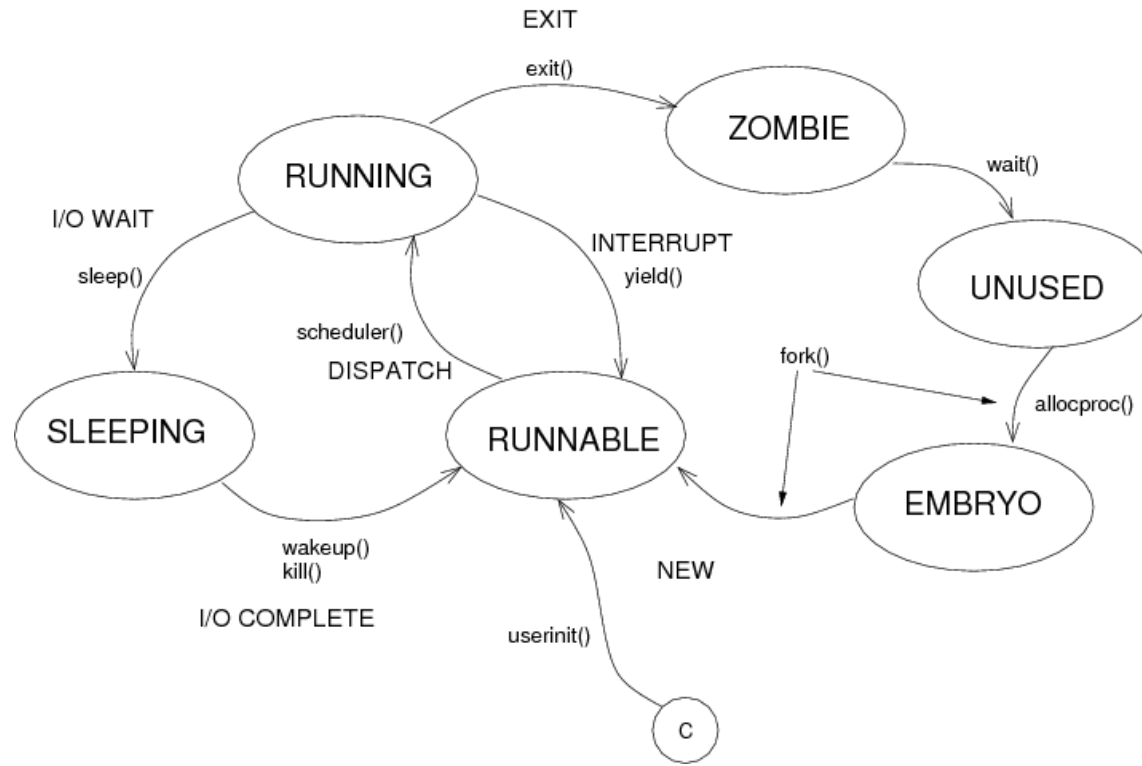
- Salida normal (voluntaria)
- Salida por error (voluntaria)
- Error fatal (involuntaria)
- Matado por otro proceso (involuntario)

Estados de un proceso

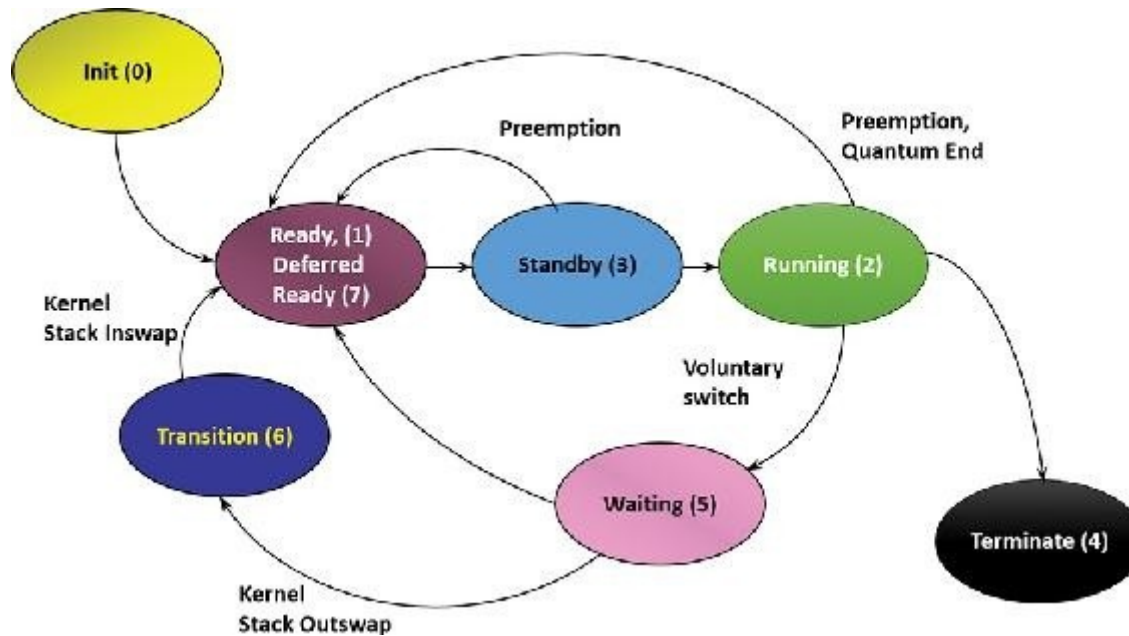
- Corriendo – Cuando está siendo ejecutado, tiene el control de la CPU
- Bloqueado - Está esperando que ocurra algún evento.
- Listo - Está esperando ser asignado a un procesador



Estados de los procesos en XV6

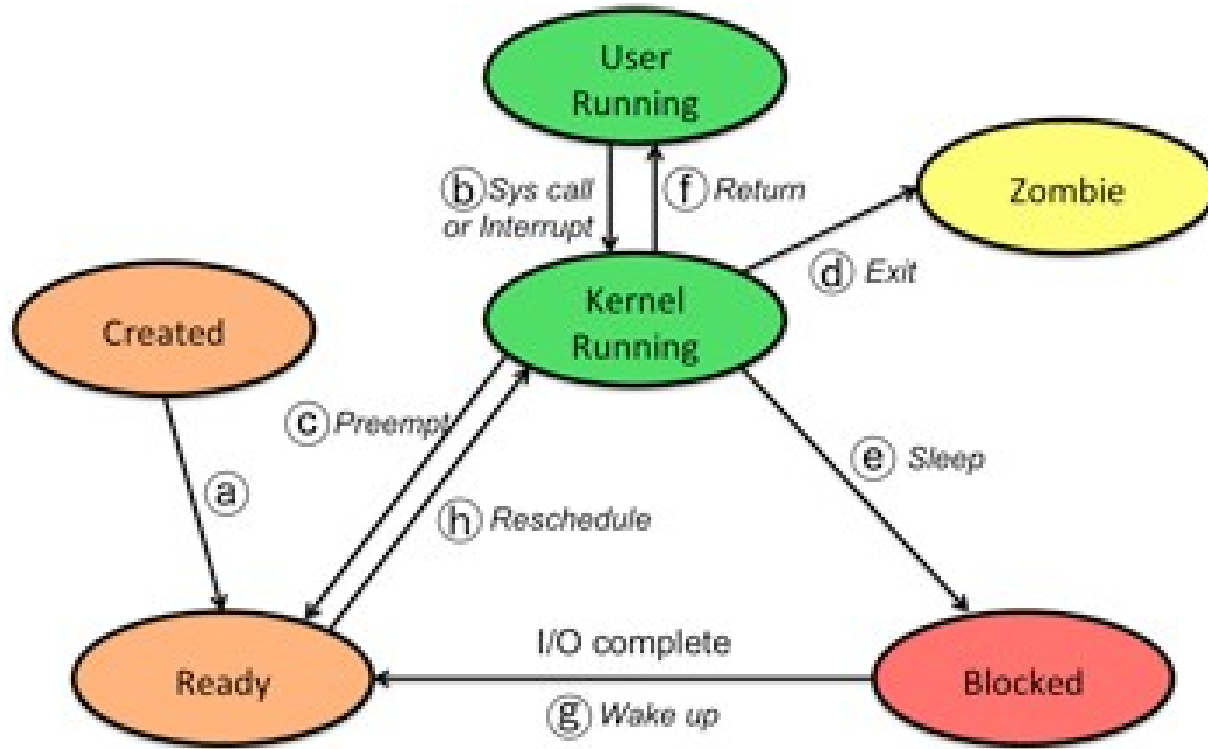


Estado de los hilos en Windows



Windows internals Fig 4.8 pag 325

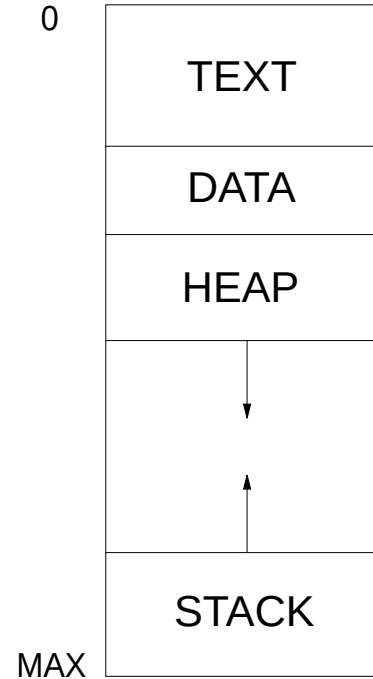
Estados de los procesos en Linux



<https://people.cs.rutgers.edu/~pxk/416/notes/04-processes.html>

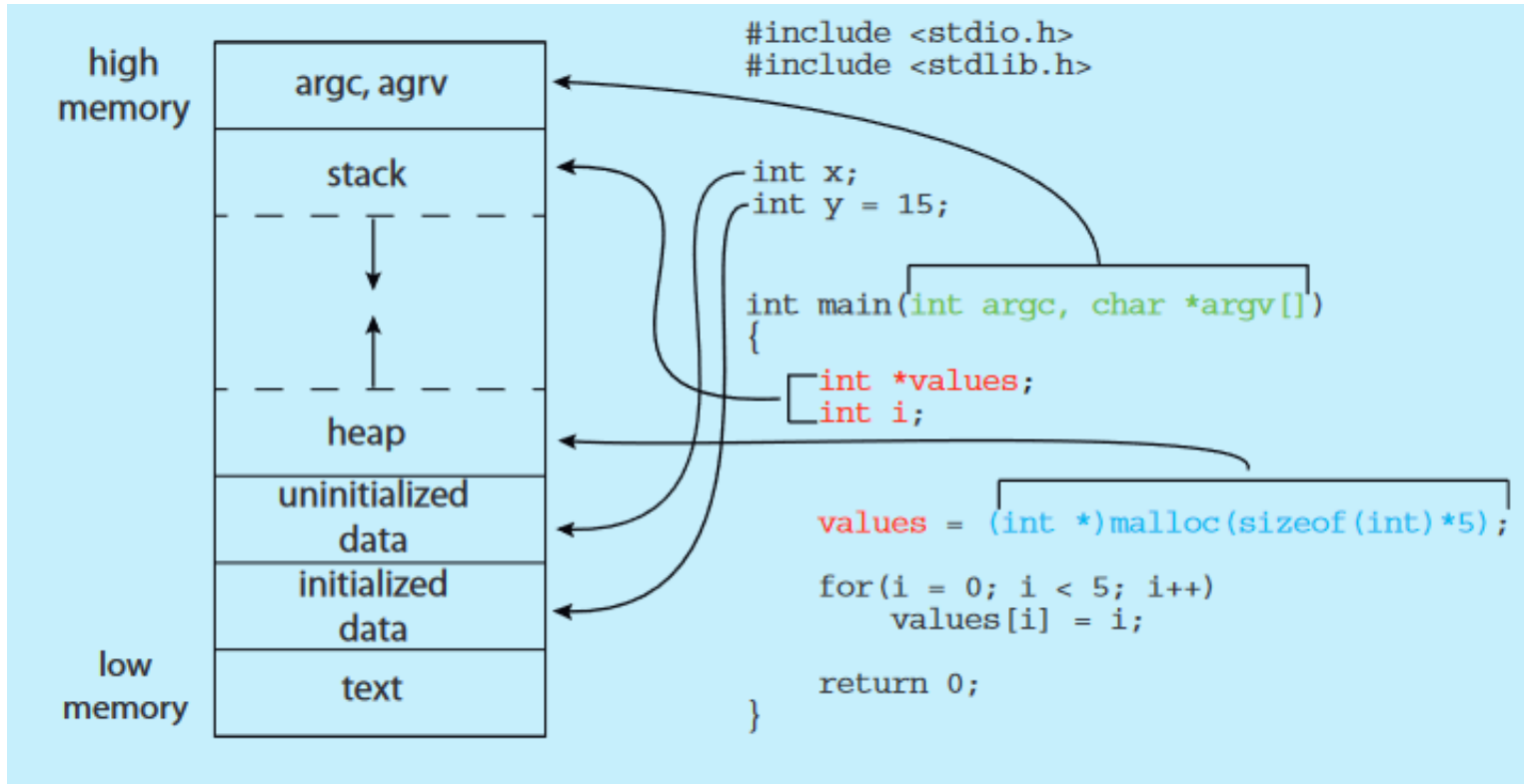
Implementación de un proceso

- Variables asociadas a un proceso
 - Contador de programa
 - Registros del procesador
 - Mapa y contenidos de la memoria
 - Otras variables



mapa de memoria usado en máquinas de 32 bits

Disposición de la memoria de un programa C



Datos del programa

- Data y BSS son secciones de memoria en un programa.
- Sección **Data**:
 - Guarda datos con valor inicial (ej: `int x = 5;`).
 - Ocupa espacio dentro del archivo binario o ejecutable.
 - Se carga directo desde el disco o memoria flash a la RAM.
- Sección **BSS** (*Block Started by Symbol*):
 - Guarda datos sin valor inicial (ej: `int y;`).
 - No ocupa espacio en el archivo binario, solo un registro con su tamaño. El sistema operativo o el código de inicio la llena de ceros al arrancar.

XV6 User memory layout



```
0246 // map the trampoline page to the highest address,
0247 // in both user and kernel space.
0248 #define TRAMPOLINE (MAXVA - PGSIZE)
0249
...
0254 // User memory layout.
0255 // Address zero first:
0256 // text
0257 // original data and bss
0258 // fixed-size stack
0259 // expandable heap
0260 // ...
0261 // SHARED_PAGE (available to all user processes)
0262 // TRAPFRAME (p->trapframe, used by the trampoline)
0263 // TRAMPOLINE (the same page as in the kernel)
0264
0265 #define TRAPFRAME (TRAMPOLINE - PGSIZE)
0266
0267 #define SHARED_PAGE (TRAPFRAME - PGSIZE)
0268
...
0929 // one beyond the highest possible virtual address.
0930 // MAXVA is actually one bit less than the max allowed by
0931 // Sv39, to avoid having to sign-extend virtual addresses
0932 // that have the high bit set.
0933 #define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
0934
```

<-- Página compartida !!! solo para esta versión

<--- Max Virtual Address = 256 Gbytes

Process Control Block (PCB)

- Contiene la información de un proceso
 - Estado (corriendo, listo, bloqueado,...)
 - Contador de programa
 - Otros registros del procesador
 - Mapa de memoria
 - Información de planificación – prioridad, ...
 - Información contable – cuando inició, tiempo acumulado, usuario, ...
 - Estado de I/O - dispositivos asignados, archivos abiertos, ...

XV6 process control block

```
2131 struct proc {
2132     struct spinlock lock;
2133
2134 // p->lock must be held when using these:
2135     enum procstate state; // Process state
2136     void *chan; // If non-zero, sleeping on
2137     int killed; // If non-zero, have been
2138                 // killed
2139     int xstate; // Exit status to be returned
2140                 // to parent's wait
2141     int pid; // Process ID
2142
2143 // wait_lock must be held when using this:
2144     struct proc *parent; // Parent process
2145
2146 // these are private to the process, so p->lock need
2147 // not be held.
2148     uint64 kstack; // Virtual address of kernel stack
2149     uint64 sz; // Size of process memory (bytes)
2150     pagetable_t pagetable; // User page table
2151     struct trapframe *trapframe; // data page for
2152                                 // trampoline.S
2153     struct context context; // swtch() here to run
2154                             // process
2155     struct file *ofile[NOFILE]; // Open files
2156     struct inode *cwd; // Current directory
2157     char name[16]; // Process name (debugging)
2158     int ctx_switches; // Number of context switches
2159     int mask; // Bitmask tracking which syscalls to
2160               // trace
2161 }
```



UNIVERSIDAD
DEL CEMA

UCEMA

Tipos de programas ejecutables

- El kernel
- Las bibliotecas
- Las aplicaciones

En linux la biblioteca está en `/lib/libc.so.6`, en windows `Msvcrt.dll`, más información:

http://en.wikipedia.org/wiki/C_standard_library

<http://msdn.microsoft.com/en-us/library/abx4dbyh.aspx>

http://en.wikipedia.org/wiki/Windows_library_files



¿Como el sistema operativo corre los procesos?

- El sistema operativo debe llevar la cuenta de cuales secciones de memoria fueron asignados a cada uno de los procesos y otras cosas más.
- Para correr un nuevo proceso, se comienza por asignarle memoria y copiar el código en esa memoria, cambiar a modo usuario y comenzar a correr por la primera dirección del programa.
- El sistema operativo mantiene un registro de cada proceso que no está corriendo.
 - La memoria asignada, el contador de programa, el puntero al stack, etc.
 - Esto se llama el contexto del proceso.
 - Suficiente información para restablecer el proceso desde donde fue detenido.

Cuando ocurre un trap

- Puede ocurrir tanto por un programa que está corriendo o bien por una señal de un dispositivo externo (Ej: una interrupción de reloj).
- Primero el sistema operativo guarda el estado del proceso que está corriendo (el contexto de proceso),
- Después se hace lo que se pide en el trap (Ej: enviar algo a la impresora),
- Finalmente se elegirá un proceso para continuar, puede ser el proceso que estaba corriendo u otro. Se carga el contexto del nuevo proceso y se regresa al modo usuario.

Jerarquía de los procesos en Unix

- Un proceso Unix nace cuando otro proceso lo crea.
 - Un proceso recientemente creado es el "hijo" (Child) del "padre" (Parent) que lo creó.
 - Cada proceso tiene exactamene un padre.
 - Un proceso puede crear cualquier número de procesos hijos.
- El proceso raiz es "init" (*) , se crea cuando el sistema operativo arranca, cada proceso puede ser traceado hasta init.
- Los procesos tienen un único número identificador de proceso (PID), este es un índice a la tabla de contextos de procesos del sistema operativo.

(*) en los sistemas linux actuales es systemd.

Creación de un proceso: fork()

- La llamada al sistema fork() crea un nuevo proceso.
- El proceso hijo es (casi) un clon exacto del proceso padre (con su propia copia del espacio de direcciones del padre)
 - comienza a correr tan pronto como retorna la función fork().
 - Entonces el padre e hijo están corriendo simultáneamente.
- ¿cómo se hace?
 - El código se comporta de manera diferente en el hijo.
- ¿cómo se puede averiguar si es el hijo o el padre?
 - Para el hijo, fork() retorna 0.
 - Para el padre, fork() retorna el PID del hijo.

Remplazando un proceso: exec()

- La llamada al sistema `exec()` (y sus variantes) reemplaza el código de un proceso por otro.
- No crea ningún proceso nuevo, el nuevo programa se especifica por el nombre del archivo que contiene el código ejecutable y sus argumentos.
- El código antiguo detiene su ejecución tan pronto como se llama a `exec()` (si se encuentra el archivo ejecutable y es posible correrlo).
- `exec()` usualmente corre después de `fork()`.

Creación de un proceso hijo

```
pid=fork();  
if (pid != 0)  
    /* do parent stuff */  
else  
    exec("/bin/child");
```

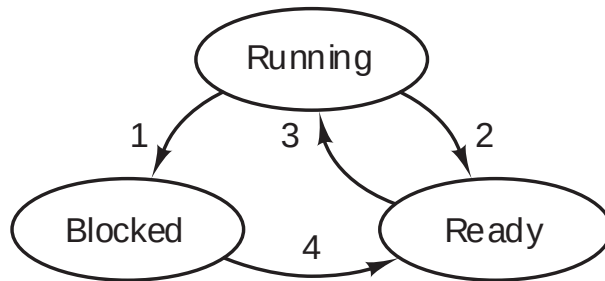
Wait()

- Habitualmente se da el caso que un proceso no tenga nada que hacer hasta que su hijo termine, Ej: lo que el interpretador de comandos hace habitualmente cuando Ud. ingresa un nuevo comando.
- wait() se bloquea hasta que el hijo termina, por lo tanto no retorna inmediatamente.
- Creación de un proceso hijo usando fork(), execve() y wait()

```
pid=fork();  
if (pid != 0)  
    wait(&status);  
else  
    if (execve("/bin/child"... )!  
=0)  
        exit(1);
```

¿qué es eso de "bloquearse"?

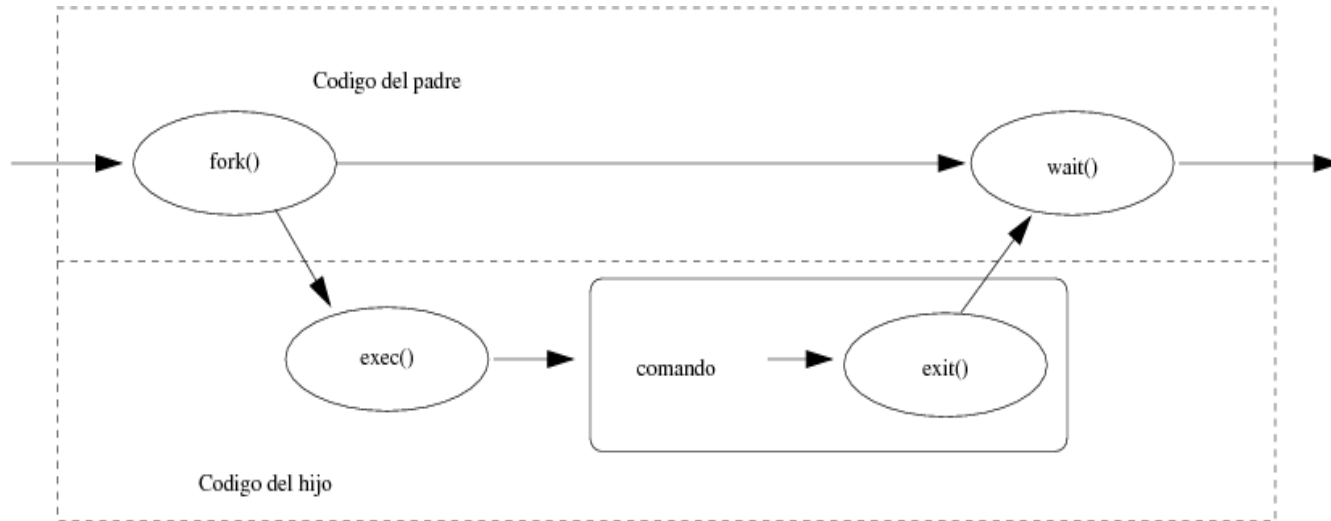
- Algunas llamadas al sistema retornan inmediatamente (Ej: fork() y exec())
- Otras se quedan esperando por algo que toma algún tiempo , por ejemplo wait(), no tiene sentido que el sistema operativo restablezca el proceso hasta que el pedido esté listo.
- Un proceso puede estar en estos tres estados básicos:



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Como el shell de unix corre los comandos:

- Cuando Ud. escribe un comando, el shell ejecuta un fork y crea un clon del mismo.
- El proceso hijo hace una llamada exec(), la cual causa que el shell se deje de ejecutar y se comience a ejecutar su comando.
- El proceso padre, todavía corriendo el shell, espera que el hijo termine.



Comunicación con los procesos: exit(), kill()

- exit() fuerza a terminar el proceso corriente.
 - Toma un valor de retorno que se puede comunicar al padre via el wait().
 - También se llama automáticamente cuando el proceso termina.
- kill() envía una señal a un proceso
 - si el proceso ha colocado un manipulador de señales (signal handler), este es llamado como si fuera una interrupción, de lo contrario se hace la acción por defecto que generalmente consiste en terminar el proceso.
 - Restricciones en el envío de señales: solo se pueden enviar señales a procesos del mismo usuario.

Entrada/Salida con read() y write()

- La entrada/salida está basada en el modelo de flujo de una secuencia de bytes.
- write() envía un flujo de bytes a algún lado.
- read() se bloquea hasta que un flujo de entrada esté listo.

Descriptores de archivo: open(), close()

- un proceso puede tener varios flujos de entrada/salida en uso en un momento determinado.
- Estas están especificadas por una estructura del kernel llamada descriptor de archivo, cada proceso tiene su propia tabla de descriptores de archivo.
 - Open() asocia un descriptor de archivo con un archivo.
 - Close() destruye un descriptor de archivo.
- La entrada estandard y la salida estandard están habitualmente asociada con una terminal.

Más sobre entrada/salida

- Es posible enganchar la salida de un programa en la entrada de otro: `pipe()`
- Es posible bloquearse hasta que uno de varios descriptores de corrientes de archivos esté listo: `select()`
- Llamadas especiales para relacionarse con la red: `sockets`, etc.